Part 6 — HBase Administration

HBase 2.6.5 on the lab cluster: one master and two RegionServers, hbase-rs-0 and hbase-rs-1. The table is user_logs. It has one column family, info, ten pre-split regions, and 1,000,000 rows. The row key is built so a burst of new events does not all land in the last region.

Table and the row key

A row key that starts with the current time, or with a user id that only goes up, writes every new row at the right-hand end of the table. One region takes all the inserts. I put a salt in front of the key so the writes spread out, and I pre-split on that salt so the regions exist before the load starts.

The key is SS|UUUUUUUU|RRRRRRRRRRRRR. SS is user_id modulo 10, written as two digits, 00 through 09. UUUUUUUU is the user id, zero padded to 8 digits. RRRRRRRRRRRRR is 9999999999999 minus the event time, so a newer event sorts first for that user. One user's events stay next to each other. Ten users with different salts go to ten different regions.

Pre-split into 10 regions

Nine split points give ten regions. The splits are the salt boundaries: 01|, 02|, through 09|. The create runs on the master from a file, because a kubectl heredoc into hbase shell was cut off and did not create the table. The script is scripts/part6-hbase/create_user_logs.sh, and the same file is on the VM at /opt/hadoop-lab/hbase/scripts/create_user_logs.sh.

```
create "user_logs", {NAME => "info", VERSIONS => 1}, {
  SPLITS => ["01|", "02|", "03|", "04|", "05|", "06|", "07|", "08|", "09|"]
}

list_regions "user_logs"
```

The master reports one active master and two servers. user_logs is ENABLED. The only column family is info, and it keeps one version of each cell.

```
hbase shell: status; describe "user_logs"

1 active master, 0 backup masters, 2 servers, 0 dead, 6.0000 average load

Table user_logs is ENABLED
NAME => 'info', VERSIONS => '1', COMPRESSION => 'NONE', BLOOMFILTER => 'ROW'
```

Figure 1. status and describe. user_logs is enabled, column family info, VERSIONS 1.

list_regions shows the ten ranges. Five are on hbase-rs-0 and five are on hbase-rs-1. Each region reports 200,000 requests. The first region runs from the start of the table to 01|, which is salt 00. The last region runs from 09| to the end of the table. Nothing is piled onto one server.

```
hbase shell: list_regions "user_logs"
```

```
Server       Start   End    Requests
hbase-rs-0   (none)  01|    200000
hbase-rs-0   01|     02|    200000
hbase-rs-1   02|     03|    200000
hbase-rs-1   03|     04|    200000
hbase-rs-0   04|     05|    200000
hbase-rs-0   05|     06|    200000
hbase-rs-1   06|     07|    200000
hbase-rs-0   07|     08|    200000
hbase-rs-1   08|     09|    200000
hbase-rs-1   09|     (none) 200000
10 rows
```

Figure 2. Ten regions, five on each RegionServer. The start and end keys are the salt boundaries.

Load

The load is a BufferedMutator in the HBase shell, not a MapReduce bulk load. The master image does not have a YARN client set up for ImportTsv, and one client is enough for a million short rows. The write buffer is 8 MB. The WAL stays on. Every 100,000 rows the script flushes and prints a line. There are 50,000 users and 20 events each. The event type cycles through pickup, dropoff, payment, cancel, and rating. The script is scripts/part6-hbase/load_user_logs.rb.

```
kubectl cp scripts/part6-hbase/load_user_logs.rb hadoop/$POD:/tmp/load_user_logs.rb
kubectl exec -n hadoop $POD -- hbase shell /tmp/load_user_logs.rb
```

The shell printed flushed 100000 through flushed 900000, then RESULT loaded=1000000 table=user_logs. count 'user_logs' walked the whole table and returned 1,000,000 rows in 53.5 seconds. The row it printed at each 200,000 mark steps through the salts two at a time, which is 100,000 rows in each salt.

```
count 'user_logs', INTERVAL => 200000
```

```
Current count: 200000, row: 01|00049991|8299999950008
Current count: 400000, row: 03|00049993|8299999950006
Current count: 600000, row: 05|00049995|8299999950004
Current count: 800000, row: 07|00049997|8299999950002
Current count: 1000000, row: 09|00049999|8299999950000
1000000 row(s)
Took 53.5078 seconds
=> 1000000
```

Figure 3. count of user_logs. 1,000,000 rows in 53.5 seconds. Each 200,000 rows covers two salts.

A get of the first row shows the key layout. Salt 00, user 00000000, reverse timestamp 8299999999999. The columns under info are user_id, event_type, ts, and amount. The ts column is the event time, 1700000000000. The HBase cell timestamp on that get is the server time of the put, 2026-10-18 15:38.

```
get "user_logs", "00|00000000|8299999999999"
```

```
COLUMN            CELL
 info:amount      value=0.00
 info:event_type  value=pickup
 info:ts          value=1700000000000
 info:user_id     value=0
1 row(s)
```

Figure 4. get of 00|00000000|8299999999999. Four columns in the info family, one row.

Major compaction

```
major_compact 'user_logs'
```

A bulk load flushes a lot of small HFiles. A read then has to open every file in the store. A major compaction rewrites one column family in a region into a single file and drops deleted cells and extra versions. I would run one after a bulk load, after a large delete or update, or when a store's file count stays high. I would not major-compact every table on a timer during the busy hour. It rewrites all of the bytes and takes IO from the live reads. Minor compaction already merges a few files on its own. Major is the full rewrite, and I run it when the file count or the delete volume calls for it.

I ran major_compact 'user_logs' from the shell. The shell returns when the request is accepted. The RegionServers do the rewrite after that. On HDFS, the info store of region d6b3b133 has a file dated 15:48, bc5fcf88, 14,314,986 bytes. That is the compacted file. A second file in the same store is dated 16:42. A flush after the compaction wrote it, so this store has two files again. That is the normal reason to compact: flushes keep adding files, and compaction folds them back.

```
hdfs dfs -ls /hbase/data/default/user_logs
```

```
Found 11 items
drwxr-xr-x  user_logs/.tabledesc
drwxr-xr-x  user_logs/34cc2960ad21cb5f7cdc42ead0a23e32
drwxr-xr-x  user_logs/90a76e722d59709b441545f26655f9bb
drwxr-xr-x  user_logs/92e9f733f2742f4ebbdd01a760c7e287
drwxr-xr-x  user_logs/9a7fc7b6b720c7c2c7014888420a7a41
drwxr-xr-x  user_logs/a01b3f47ae2c1bbb70b6f5535f8e6eed
drwxr-xr-x  user_logs/c12b94bd2b733b52516b871119176fb3
drwxr-xr-x  user_logs/d6b3b133a15a8f6bb4843ed87b16e562
drwxr-xr-x  user_logs/e3dbd15212c4820d2aaf34d056677a1c
drwxr-xr-x  user_logs/e9df73de3521dd93a3bcabb44338ec84
drwxr-xr-x  user_logs/f112e60302be0b983e129bc093f21f33
```

Figure 5. The table directory on HDFS. .tabledesc plus ten region directories, one per pre-split.

```
hdfs dfs -ls .../d6b3b133a15a8f6bb4843ed87b16e562/info
```

```
-rw-r--r--  2 root supergroup  14314986  2026-10-18 15:48  bc5fcf88f6f243949c03d3b32ff06d0d
-rw-r--r--  2 root supergroup  11492020  2026-10-18 16:42  1d0e553368c04328a2f381f0e40ddc46
```

Figure 6. The info store for region d6b3b133. The 15:48 file is the major compaction. The 16:42 file is a later flush.

The other nine stores look the same shape. Most have the 15:48 compaction file plus one smaller file from about 16:40. Region 92e9f733 has a single file, dated 16:43, of 25,835,683 bytes. Each region is about 26 MB. The HFiles are replication 2, which is the cluster default. The taxi dataset was the path set to replication 3. This table was not.

hbck

This is HBase 2.6.5, r0255064e5ed1b34c55a5392cd65829fac7a7ab1f. On 2.x, hbase hbck with no extra jar still launches the old HBaseFsck. The check for this version is hbck2. I used hbase-hbck2-1.3.0.jar and the read-only commands filesystem, reportMissingRegionsInMeta, and extraRegionsInMeta for default:user_logs. There is no report command in hbck2 1.3.0.

```
hbase hbck -j /tmp/hbase-hbck2.jar filesystem default:user_logs
```

hbck2 did reach ZooKeeper. The log shows a session on zk-2. The command then died with UnsupportedFileSystemException: No FileSystem for scheme "hdfs". The same exception stopped the other two commands. The HBase image on the master does not put the HDFS client on the classpath that hbck2 uses. Adding hadoop-hdfs, hadoop-common, and guava by hand moved the error to the next missing class. That is a client classpath gap. It is not a hole in user_logs.

```
hbase hbck -j /tmp/hbase-hbck2.jar filesystem default:user_logs

Session establishment complete on server zk-2.zk-hs...:2181, session id = 0x3000999c17f000c
Exception in thread "main" org.apache.hadoop.fs.UnsupportedFileSystemException: No FileSystem for scheme "hdfs"
    at org.apache.hadoop.fs.FileSystem.getFileSystemClass(FileSystem.java:3239)
    at org.apache.hbase.FileSystemFsck.<init>(FileSystemFsck.java:44)
    at org.apache.hbase.HBCK2.doCommandLine(HBCK2.java:1258)
    at org.apache.hbase.HBCK2.main(HBCK2.java:1561)
```

Figure 7. hbck2 filesystem for default:user_logs. ZooKeeper answered. The HDFS client class is missing, so the check stops.

What I can say about the table without that report: describe says ENABLED, list_regions shows all ten regions hosted, count returns 1,000,000, and HDFS has an info store with HFiles under every region directory. A clean hbck2 filesystem line would be the last check. The classpath is what is in the way, not the regions.

Where to look

Scripts on the VM: /opt/hadoop-lab/hbase/scripts/create_user_logs.sh, load_user_logs.sh, and load_user_logs.rb.

The hbck2 jar and the failed report are on the VM at /opt/hadoop-lab/hbase/scripts/hbase-hbck2.jar and hbck-user_logs.txt.

Table files: hdfs://namenode:9000/hbase/data/default/user_logs

Master UI port on the service is 16010. It is not port-forwarded on the VM right now.